All the code up to now has completely disregarded exceptions. Although it might be nice to imagine a world in which everything worked all the time, this isn't actually the case. Sometimes disks fill up, sometimes what you're looking for just isn't there, sometimes the network fails, and sometimes the database goes down. If you were performing the operation in the thread that needed the result, the code could just report an error with an exception, so it would be unnecessarily restrictive to require that everything go well just because you wanted to use a `std::packaged_task` or a `std::promise`. The C++ Standard Library therefore provides a clean way to deal with exceptions in such a scenario and allows them to be saved as part of the associated result.

4.2.4. Saving an exception for the future

Consider the following short snippet of code. If you pass in `-1` to the `square_root()` function, it throws an exception, and this gets seen by the caller:

```
double square_root(double x)
{
    if(x<0)
    {
        throw std::out_of_range("x<0");
    }
    return sqrt(x);
}
```

Now suppose that instead of just invoking `square_root()` from the current thread,

```
double y=square_root(-1);
```

you run the call as an asynchronous call:

```
std::future<double> f=std::async(square_root,-1);
double y=f.get();
```

It would be ideal if the behavior was exactly the same; just as `y` gets the result of the function call in either case, it would be great if the thread that called `f.get()` could see the exception too, just as it would in the single-threaded case.

Well, that's exactly what happens: if the function call invoked as part of `std::async` throws an exception, that exception is stored in the future in place of a stored value, the future becomes *ready*, and a call to `get()` rethrows that stored exception. (Note: the standard leaves it unspecified whether it is the original exception object that's rethrown or a copy; different compilers and libraries make different choices on this matter.) The same happens if you wrap the function in a `std::packaged_task`—when the task is invoked, if the wrapped function throws an exception, that exception is stored in the future in place of the result, ready to be thrown on a call to `get()`.

Naturally, `std::promise` provides the same facility, with an explicit function call. If you wish to store an exception rather than a value, you call the `set_exception()` member function rather than `set_value()`. This would typically be used in a `catch` block for an exception thrown as part of the algorithm, to populate the promise with that exception:

```
extern std::promise<double> some_promise;
```